

Share

 Comment Star

...

License Plate Recognition

[lizzzafomenko](#)

Created on November 22 | Last edited on November 22

Для решения задачи я использовала обычную CRNN. Мне показалось правильным использовать претренин, так как данных было не очень много. Поэтому я нашла готовую реализацию у прекрасного человека: <https://github.com/GitYCC/crnn-pytorch>

Модель обучили на датасете Synth90k, который состоит из 9 миллионов картинок, на которых находится 90 тысяч английских слов. чекпоинт лежит в том же репозитории.

На самом деле я сделала очень немного изменений. Во-первых, в качестве словаря я использовала только те символы, которые встречаются в русских номерах (10 цифр + 12 букв). Во-вторых, я изменила последний dense слой, чтобы его выходной размер совпадал с размером нового словаря. В последнем запуске я добавила небольшой dropout_rate = 0.2 в BLSM слой, что привело к очень маленькому, практически незначительному, улучшению на тесте.

Исходная реализация использовала в качестве лосса CTCLoss, я оставила его в нетронутым виде. Изменила я лишь оптимизатор с RMSprop на AdamW, а также последовательно подобрала оптимальные размер батча (64) и lr, который оказался равным 1e-3. Замечу, что я использовала clip_grad = 0.2, поэтому, возможно, можно было использовать такой lr. Но на самом деле на графике train_loss прекрасно видно, что первый запуск (который показан красно-розовым) сеть безбожно взрывалась, хотя это не помешало ей сразу побить последний из обязательных порогов. После увеличения размера батча и clip_grad такие мощные взрывы исчезли (избавление от них совсем приводило к очень быстрому переобучению)

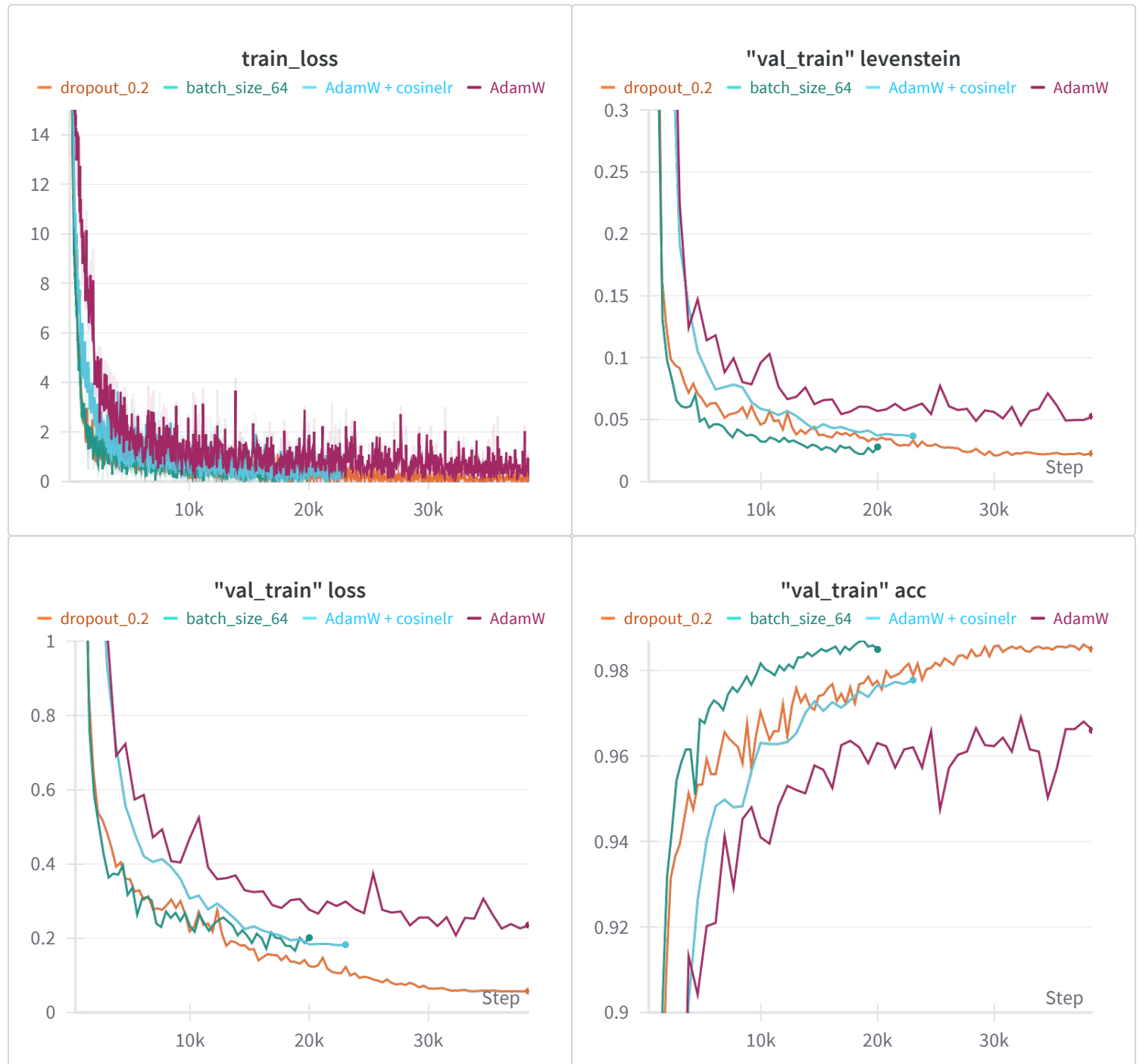
Я разделила данные на train и val, в val были добавлены первые 500 картинок из simple и complex train набора. Здесь я **выделю**, что на графиках представлены только те запуски, которые привели к улучшению результата, и были использованы на predict_test. При этом, так как данных немного, для сдачи хороших идей я ставила файнтюнинг на всех имеющихся данных (это помогало немного повысить скор), поэтому смысл валидации терялся (на графиках совсем не видно переобучения!), поэтому я назвала графики "val_train". В общем, они не отражают реальной валидации (там было заметно переобучение на +30 эпохе, поэтому для финальных запусков я сливала train_dataset и val_dataset и ставила на 35 эпох).

Сожалею, что у меня не сохранились запуски с адекватными значениями :(

Многие мои сабмиты проверяли самый странный набор идей. Например, мне очень хотелось, чтобы сработал TestTimeTraining, так как картинки по факту шли из двух разных распределений (simple и complex), и я хотела как-то это использовать, но TTT ситуацию не улучшил. Еще я пробовала использовать LoRA для файнтюнинга, но это было заведомо плохой идеей (так как готовые пакеты умеют работать только с `nn.Linear` или `nn.Conv2d(group=1)`, а таких слоев в модели было немного).

Какой вывод я бы сделала: на самом деле почти все 100% успеха случились благодаря самой модели и очень удачному претрейну. Как бы я ни экспериментировала, на пабlike у меня только пару раз получились решения ниже `baseline20`, но все еще выше `baseline15`. Из того что я заметила, модель могла путать букву О с цифрой 0, и я полагаю, что существуют методы решения этой проблемы, до которых у меня не дошли руки. Также в этой задаче были бессмысленны некоторые аугментации (слишком большие повороты, Horizontal / Vertical Flips), что вполне логично.

Graphics



Created with ❤️ on Weights & Biases.

http://localhost:8385/lizzafomenko/license_plates/reports/License-Plate-Recognition--VmldzozMw